Bash Scripting — Complete Notes

Shell automation · Variables, conditionals, loops, functions & more

A reader-friendly, all-in-one reference for Bash scripting.

---

1. 🐚 What is Bash

Bash (**Bourne Again SHell**) is the default command-line shell on most Linux systems and a scripting language for **automating tasks**: gluing commands together, file manipulation, deployments, CI/CD steps, backups, and system administration. A Bash **script** is just a text file of shell commands run top to bottom.

> 💡 **Mental model:** Bash is **automation glue** — it orchestrates other programs (`grep` , `curl` , `aws` …), wiring their inputs/outputs together. It shines for short automation; reach for Python when logic gets complex.

Common uses: CI/CD pipeline steps, deploy/build scripts, cron jobs, log processing, environment setup, and one-off sysadmin tasks.

---

2. ⚙️ Script Basics (Shebang & Running)

```
#!/usr/bin/env bash      # shebang — tells the OS to run this with bash
echo "Hello, world"
```

- The **shebang** (`#!`) on line 1 picks the interpreter; `#!/usr/bin/env bash` is portable.
- Make it executable and run it:

```
chmod +x script.sh      # add execute permission
./script.sh             # run it
bash script.sh          # or run explicitly with bash
```

- `# comment` — everything after `#` is ignored. Run with `bash -x script.sh` to **trace** each command (debugging).

---

3. 📦 Variables & Quoting

```
name="Alice"            # NO spaces around = ; assignment
echo "$name"            # use with $ ; quote to be safe
echo "${name}_suffix"   # braces disambiguate
readonly PI=3.14        # constant
export PATH="$PATH:/opt/bin"   # export to child processes
```

Quoting (the #1 source of bugs):

Quote	Behavior
`"double"`	Expands `$variables` and `$(commands)` — **use by default**
`'single'`	Literal — no expansion at all
`` `backticks` ``	Old command substitution — prefer `$( )`

> ⚠️ **Always quote your variables** (`"$var"`). Unquoted, a value with spaces or globs splits into multiple words — the classic source of broken scripts.

---

4. 🔢 Command Substitution & Arithmetic

```
today=$(date +%F)           # capture command output
files=$(ls | wc -l)
echo "There are $files files as of $today"

count=$(( 3 + 4 * 2 ))      # arithmetic: 11
(( count++ ))               # increment
if (( count > 10 )); then echo big; fi
```

- `$( ... )` — **command substitution** (preferred over backticks; nests cleanly).
- `$(( ... ))` — **integer arithmetic**. For floats use `bc` or `awk`.

---

5. 🔀 Conditionals

```
if [[ "$age" -ge 18 ]]; then
  echo "adult"
elif [[ -z "$age" ]]; then
  echo "no age given"
else
  echo "minor"
fi

# case
case "$1" in
  start) echo "starting" ;;
  stop)  echo "stopping" ;;
  *)     echo "usage: $0 {start|stop}" ;;
esac
```

`[[ ]]` test operators:

Numbers	Strings	Files
`-eq -ne -lt -le -gt -ge`	`=` `!=` `-z` (empty) `-n` (non-empty)	`-f` (file) `-d` (dir) `-e` (exists) `-r/-w/-x` (perms)

> Prefer `[[ ]]` (Bash) over `[ ]` (POSIX `test`) — it's safer with strings, supports `&&` / `||` and `=~` regex, and doesn't need as much quoting.

---

6. 🔁 Loops

```
for f in *.log; do           # iterate files/words
  echo "processing $f"
done

for i in {1..5}; do echo "$i"; done      # range
for ((i=0; i<5; i++)); do echo "$i"; done # C-style

while read -r line; do         # read a file line by line
  echo "$line"
done < input.txt

until ping -c1 host &>/dev/null; do sleep 1; done   # until success
```

- `break` exits a loop; `continue` skips to the next iteration.
- `read -r` prevents backslash mangling — always use `-r`.

---

7. 🧩 Functions

```
greet() {
  local name="$1"           # local — scope to the function
  echo "Hello, $name"
  return 0                  # exit status (0 = success), not a value
}

greet "Bob"                 # call it
result=$(greet "Carol")     # "return" data via stdout + capture
```

- Functions **return an exit status** (0–255), not arbitrary values — to return data, `echo` it and capture with `$( )`.
- Use `local` for variables so they don't leak into the global scope.

---

8. 🎰 Arguments & Special Variables

Variable	Meaning
`$0`	Script name
`$1, $2, …`	Positional arguments
`$#`	Number of arguments
`$@`	All args (as separate words — quote it: `"$@"`)
`$*`	All args (as one string)
`$?`	Exit status of the **last** command
`$$`	Current script's PID
`$!`	PID of the last background command

```
if [[ $# -lt 1 ]]; then
  echo "usage: $0 <name>" >&2
  exit 1
fi
for arg in "$@"; do echo "$arg"; done
```

---

9. 📤 I/O & Redirection

Operator	Does
`>`	Redirect stdout to a file (overwrite)
`>>`	Append stdout to a file
`<`	Read stdin from a file
`2>`	Redirect stderr
`2>&1`	Send stderr to wherever stdout goes
`&>file`	Both stdout + stderr to a file
`|`	Pipe stdout of one command into stdin of the next

```
command > out.log 2>&1        # capture everything
grep ERROR log | wc -l        # pipe
cat <<'EOF' > config.txt      # here-document (literal block)
key=value
host=localhost
EOF
echo "data" | tee file.txt     # write to file AND stdout
```

> `/dev/null` is the bit-bucket: `cmd &>/dev/null` discards all output. `1` = stdout, `2` = stderr.

---

10. ✂️ Parameter Expansion & Strings

Powerful built-in string manipulation (no external tools needed):

```
file="report.txt"
echo "${#file}"          # length: 10
echo "${file%.txt}"      # strip suffix → report
echo "${file##*.}"       # extension → txt
echo "${file/report/summary}"   # replace → summary.txt
echo "${name:-default}"  # use 'default' if name is unset/empty
echo "${name:=def}"      # assign default if unset
echo "${path:0:4}"       # substring
echo "${var^^}"          # uppercase  ${var,,} = lowercase
```

Pattern	Meaning
`${v:-x}`	Default if unset/empty
`${v:?msg}`	Error & exit if unset
`${v#pat}` / `${v##pat}`	Trim shortest/longest **prefix**
`${v%pat}` / `${v%%pat}`	Trim shortest/longest **suffix**
`${v/a/b}` / `${v//a/b}`	Replace first / all

11. 🧰 Arrays

```
arr=(apple banana cherry)     # indexed array
echo "${arr[0]}"              # apple
echo "${arr[@]}"             # all elements
echo "${#arr[@]}"            # length: 3
arr+=(date)                   # append
for x in "${arr[@]}"; do echo "$x"; done

declare -A map                # associative array (Bash 4+)
map[name]="Alice"; map[role]="dev"
echo "${map[name]}"
for k in "${!map[@]}"; do echo "$k=${map[$k]}"; done
```

> Always expand arrays as `"${arr[@]}"` (quoted) to preserve elements with spaces.

12. 🛡️ Exit Codes & Error Handling

- Every command returns an **exit code**: **0 = success**, non-zero = failure (stored in `$?`).
- Make scripts **fail safely** with the "unofficial strict mode":

```
#!/usr/bin/env bash
set -euo pipefail
# -e  exit on any command failure
# -u  error on use of an unset variable
# -o pipefail  a pipeline fails if ANY stage fails

trap 'echo "error on line $LINENO" >&2' ERR   # run on error
trap 'rm -f "$tmp"' EXIT                      # cleanup on exit

cmd1 && cmd2        # run cmd2 only if cmd1 succeeded
cmd1 || echo "fail" # run only if cmd1 failed
mycommand || exit 1 # propagate failure
```

> `set -euo pipefail` + `trap` is the standard safety harness — without it, scripts silently continue after errors and use empty unset variables.

---

13. 🔧 Essential Text Tools

Bash orchestrates these constantly:

Tool	Use
`grep`	Search lines by pattern (`grep -r 'ERROR' .`)
`sed`	Stream edit / substitute (`sed 's/old/new/g'`)
`awk`	Column/field processing (`awk '{print $1}'`)
`cut` / `sort` / `uniq` / `wc`	Slice / sort / dedupe / count
`find`	Locate files (`find . -name '*.log' -mtime +7`)
`xargs`	Turn input into args (`find . -name '*.tmp'
`tr`	Translate/delete chars
`curl` / `jq`	HTTP + JSON parsing

```
# count error lines per file, top 5
grep -c ERROR *.log | sort -t: -k2 -nr | head -5
```

---

14. ✅ Best Practices

- Start with **`set -euo pipefail`** and a `trap` for cleanup.
- **Always quote** variables (`"$var"` , `"$@"` , `"${arr[@]}"`).
- Prefer **`[[ ]]`** over `[ ]` , and **`$( )`** over backticks.
- Use **`local`** in functions; check args (`$#`) and `exit 1` with a usage message on bad input.
- Write errors to **stderr** (`>&2`); use meaningful **exit codes**.
- Run **`shellcheck`** (a linter) on every script — it catches the common footguns.
- Use `read -r` ; avoid parsing `ls` (use globs/ `find`); quote command substitutions.
- Keep it small — if it grows complex logic/data structures, **switch to Python**.

---

15. 🧠 Quick Mental Model

- **Bash = automation glue** that orchestrates other commands; great for short scripts, not heavy logic.
- **Shebang** + `chmod +x` to run; comment with `#` ; trace with `bash -x` .

- **Quote everything** (`"$var"`) — unquoted values split on spaces/globs (the #1 bug).
- `$( )` captures output; `$(( ))` does integer math; `[[ ]]` tests conditions.
- Loops: `for` / `while read -r` / `until`. Functions return an **exit status**; "return" data via stdout.
- **Special vars:** `$1` / `$@` / `$#` / `$?` / `$0` . Redirect with `>` `>>` `2>&1` `|` ; `&>/dev/null` discards.
- **Parameter expansion** (`${v%.txt}` , `${v:-default}`) does strings without external tools.
- `set -euo pipefail` + `trap` = safety harness; **0 = success**; run **shellcheck**.

---

16. ❓ Common Interview Questions

Rapid-fire questions interviewers ask about Bash:

- **Q: What does the shebang do?** — The `#!` line picks the interpreter that runs the script (e.g. `#!/usr/bin/env bash`).
- **Q: Single vs double quotes?** — Double quotes expand `$variables` / `$(...)` ; single quotes are fully literal (no expansion).
- **Q: Why quote variables?** — Unquoted values undergo word-splitting and globbing, so a value with spaces breaks into multiple arguments.
- **Q: What does `$?` hold?** — The exit status of the last command (0 = success, non-zero = failure).
- **Q: `[ ]` vs `[[ ]]` ?** — `[[ ]]` is the Bash test: safer with strings/empty vars, supports `&&` , `||` , and `=~` regex.
- **Q: What does `set -euo pipefail` do?** — Exit on error (`-e`), error on unset variables (`-u`), and fail a pipeline if any stage fails (`pipefail`).
- **Q: `$@` vs `$*` ?** — `"$@"` expands each argument as a separate quoted word (almost always what you want); `"$*"` joins them into one string.
- **Q: How does a function "return" a value?** — `return` sets only the exit status (0–255); to return data, `echo` it and capture with `$(...)` .
- **Q: How do you redirect stderr to stdout?** — `2>&1` (order matters: `> file 2>&1`).
- **Q: Command substitution syntax?** — `$(command)` (preferred) or legacy backticks; captures the command's stdout.
- **Q: How do you debug a script?** — `bash -x script.sh` to trace, and run `shellcheck` to lint for common mistakes.
- **Q: Bash vs Python — when?** — Bash for short command orchestration/glue; Python when you need real data structures, complex logic, or maintainability.

Notes compiled as a quick reference for Bash scripting.